

1) **Понятие системы счисления.**

Система счисления — совокупность принципов и правил, позволяющих представить число. Позиционная система счисления — система, в которой значение цифры зависит от её позиции в числе, умноженной на весовой коэффициент, определяемый основанием системы (p). Основание — количество различных цифр, используемых в системе (например, 2 для двоичной, 10 для десятичной). Число записывается как последовательность цифр, где целая и дробная части разделены запятой, а веса цифр увеличиваются в p раз для каждой позиции слева направо.

2) **Перевод чисел из одной системы счисления в другую.**
Как по кайфу.

3) **Представление чисел с фиксированной и плавающей запятой в ЭВМ.**

Числа с плавающей запятой представляются как $N = \pm m \cdot 10^{\pm p}$ (в двоичной системе основание 2), где m — мантисса ($0,1 \leq |m| < 1,0$), p — порядок (целое число). Числа с фиксированной запятой: Упоминаются в контексте двоично-десятичных кодов (D-кодов), где числа представляются в формате с фиксированной запятой, но подробного описания их представления нет. Применяются D-коды (например, D8421 или D8421+3).

4) **Форматы данных, прямой, обратный, дополнительный код.**

- 1) Прямой код: Первый разряд кодирует знак (0 — положительное, 1 — отрицательное), остальные разряды — само число.
- 2) Обратный код: Получается инверсией всех разрядов числа, знаковый разряд становится 1 для отрицательных чисел.
- 3) Дополнительный код: Формируется инверсией всех разрядов числа и добавлением 1 к младшему разряду или инверсией разрядов до последней единицы, оставляя её. При сложении числа и его дополнительного кода получается 0 с переносом из старшего разряда. Дополнительный код удобен для операций, так как не требует дополнительных действий.

5) **Выполнение операции алгебраического сложения в ЭВМ.**

В прямом коде сложение требует анализа знаков операндов, зануления знаков и, при необходимости, присвоения знака результату. Для вычитания ($A - B$) используется дополнительный код: $A - B = A + [B]_{\text{доп}}$, где $[B]_{\text{доп}}$ — дополнительный код B , независимо от знака B . В обратном коде при переносе из старшего разряда добавляется 1 к младшему разряду. Переполнение

отслеживается: для положительных чисел — появление 1 в знаковом разряде, для отрицательных — 0. Модифицированные коды с двумя знаковыми разрядами (00 — положительное, 11 — отрицательное) упрощают отслеживание переполнений.

6) **Арифметика чисел с плавающей запятой. Погрешности представления.**

Числа с плавающей запятой представляются как $N = \pm m \cdot 2^{\pm p}$, где m — мантисса ($0,1 \leq |m| < 1,0$), p — порядок (целое число). Арифметические операции включают сложение порядков при умножении ($pC = pA + pB$) и вычитание при делении ($pC = pA - pB$). Нормализация результата обязательна: мантисса сдвигается, чтобы $0,1 \leq |m| < 1,0$, с корректировкой порядка. **Погрешности:** Упоминается необходимость округления результата (например, прибавление $2^{-(n+1)}$ к сумматору при умножении) для избежания накопления систематической ошибки отрицательного знака. Подробного описания других видов погрешностей нет.

7) **Умножение двоичных чисел.**

Умножение двоичных чисел в формате с плавающей запятой включает умножение мантисс и сложение порядков ($pC = pA + pB$). Знак результата определяется как $T = \text{ЗН}(A) \oplus \text{ЗН}(B)$. Алгоритмы:

- 1) Младшими разрядами вперед: Мантиссы загружаются в регистры, сумматор обнуляется. Если младший разряд множителя равен 1, сумматор добавляет множимое. Выполняются сдвиги сумматора и множителя вправо, повторяется $n - 1$ раз.
- 2) Старшими разрядами вперед: Аналогично, но сдвиги влево, начиная со старших разрядов.
- 3) В дополнительных и обратных кодах учитываются знаки и корректировки.

8) **Методы ускорения выполнения операции умножения.**

Ускоренные методы умножения включают:

- 1) Анализ двух разрядов множителя: За один цикл обрабатываются два разряда, сокращая количество циклов вдвое. Используются условные операции сложения/вычитания в зависимости от комбинации разрядов.
- 2) Метод Бутта: Учитывает последовательности единиц и нулей в множителе, заменяя серии сложений вычитаниями и сдвигами.

- 3) Метод Уоллеса: Параллельное сложение частичных произведений с использованием дерева сумматоров, минимизирующее задержки.
- 9) **Деление двоичных чисел в прямых кодах.**
Деление в прямых кодах выполняется с подвижным или неподвижным сумматором:
- 1) С подвижным сумматором: Делимое в сумматоре, делитель в регистре. Сумматор сдвигается влево, вычитается делитель, формируется разряд частного. С восстановлением остатка (добавление делителя при отрицательном остатке) или без.
 - 2) С неподвижным сумматором: Делитель сдвигается вправо, сумматор вычитает/добавляет делитель в зависимости от знака остатка, разряд частного заносится в регистр.
Порядок результата: $pC = pA - pB$. Знак: $T = ZH(A) \oplus ZH(B)$.
- 10) **Деление двоичных чисел в дополнительных кодах.**
Деление в дополнительных кодах аналогично прямым кодам, но использует дополнительный код для отрицательных чисел. Операции вычитания реализуются как сложение с дополнительным кодом делителя. Алгоритм с неподвижным сумматором: делимое в сумматоре, делитель в регистре сдвигается вправо, сумматор добавляет/вычитает делитель в зависимости от знака. Частное формируется в регистре, порядок корректируется ($pC = pA - pB$). Знак результата определяется по XOR знаков операндов.
- 11) **Ускоренные методы операции деления.**
Ускоренные методы деления включают:
- 1) Анализ одного разряда после запятой: Проверяется дополнительный разряд мантиссы делителя, что позволяет сократить количество циклов деления за счёт предсказания следующего разряда частного.
 - 2) Анализ двух разрядов после запятой: Анализируются два разряда делителя, что уменьшает число итераций вдвое, используя условные операции сложения/вычитания в зависимости от комбинации разрядов.
 - 3) Метод Ньютона-Рафсона: Итеративное вычисление обратного значения делителя с последующим умножением на делимое для получения частного.
 - 4) Метод CRT (сравнительно-рекурсивный): Использует рекурсивное разбиение делителя для ускорения вычислений.

12) Извлечение корня из двоичных чисел.

Извлечение квадратного корня из двоичных чисел реализуется с использованием алгоритма с подвижным сумматором. Делимое загружается в сумматор, а результат (корень) формируется в регистре. Алгоритм:

- 1) Вычитание последовательных нечётных чисел (1, 3, 5, ...) из делимого.
- 2) Сдвиг сумматора влево, добавление разряда в регистр результата.
- 3) Проверка знака остатка для определения очередного разряда корня. Знак результата всегда положительный, порядок корректируется как $pC = pA/2$. Нормализация результата обязательна.

13) Двоично-десятичные коды (D-коды), их разновидности, области применения.

Двоично-десятичные коды (D-коды) представляют десятичные цифры (0–9) в виде 4-битных двоичных комбинаций. Разновидности:

- 1) D8421: Каждая десятичная цифра кодируется весами 8, 4, 2, 1 (например, $5 = 0101$).
- 2) D8421+3: Код D8421 с добавлением 3 (например, $5 = 0101 + 0011 = 1000$).
Области применения: обработка финансовых данных, интерфейсы ввода-вывода, где требуется совместимость с десятичной системой.

14) Особенности выполнения операции сложения в D-кодах.

Сложение в D-кодах (например, D8421) выполняется побитно, как в двоичной арифметике, с последующей десятичной коррекцией. Если сумма в группе из 4 бит превышает 9 (или перенос из старшего разряда равен 1), добавляется 0110 (6 в двоичной системе) для коррекции результата в пределах 0–9. Пример: $5 (0101) + 6 (0110) = 1011$, добавляем 0110, получаем 0001 с переносом 1 (результат 11).

15) Получение дополнительного кода чисел в D-кодах.

Дополнительный код в D-кодах (например, D8421) для отрицательного числа получается следующим образом:

- 1) Инверсия всех разрядов числа (замена 0 на 1 и наоборот).
- 2) Добавление 1 к младшему разряду. Или: инверсия всех разрядов до последней единицы, оставляя последнюю единицу без изменений. Это обеспечивает корректное представление отрицательных чисел для операций сложения/вычитания.

16) Операция умножения чисел в D-кодах.

Умножение в двоично-десятичных кодах (D-кодах, например, D8421) выполняется двумя методами:

- 1) Младшей секцией вперед: Мантисса множителя проверяется по группам (4 бита), если группа ненулевая, добавляется множимое в сумматор, затем выполняется сдвиг вправо. После каждого шага проводится десятичная коррекция (добавление 0110, если сумма > 9).
- 2) Старшей секцией вперед: Аналогично, но проверка начинается со старших групп, сдвиг влево. Требуется десятичная коррекция для каждой группы. Знак результата: $T = 3H(A) \oplus 3H(B)$. Порядок: $pC = pA + pB$.

17) Операция деления чисел в D-кодах.

Деление в D-кодах выполняется с подвижным или неподвижным сумматором:

- 1) С подвижным сумматором: Делимое в сумматоре, делитель в регистре. Вычитание делителя с десятичной коррекцией (добавление 0110, если результат > 9), сдвиг сумматора влево. С восстановлением остатка (добавление делителя при отрицательном остатке) или без.
- 2) С неподвижным сумматором: Делитель сдвигается вправо, сумматор вычитает/добавляет делитель с десятичной коррекцией. Частное формируется в регистре, порядок: $pC = pA - pB$, знак: $T = 3H(A) \oplus 3H(B)$.

18) Бинарные отношения, способы задания бинарных отношений.

Бинарное отношение — это двуместная логическая функция двузначной логики с одинаковыми алфавитами для вхождений, заданная как $r = (X^2, \{0,1\}, \tilde{R})$, где X — алфавит вхождений, $\tilde{R} \subset X^2 \times \{0,1\}$ — закон (график). Способы задания:

- 1) Перечисление элементов множества $\tilde{R}$ (тройки (x, y, r)).
- 2) Указание области истинности $A = \{(x, y) \in X^2 : r(x, y) = 1\}$ и, при необходимости, области ложности $\bar{A}$.
- 3) Двоичная квадратная матрица $n \times n$, где $n = |X|$, с элементами 0 и 1 (1, если $(x_i, x_j) \in A$).
- 4) Графическое представление (граф): вершины — элементы X , дуги — пары $(x_i, x_j) \in A$.
- 5) Аналитический способ (с использованием формул, например, через операции над отношениями).

19) Свойства бинарных отношений.

Свойства бинарных отношений включают:

- 1) Рефлексивность: $\forall x \in X, (x, x) \in A$.
- 2) Антирефлексивность: $\forall x \in X, (x, x) \notin A$.
- 3) Симметричность: $\forall x, y \in X, (x, y) \in A \Rightarrow (y, x) \in A$.
- 4) Антисимметричность: $\forall x, y \in X, (x, y) \in A \wedge (y, x) \in A \Rightarrow x = y$.
- 5) Транзитивность: $\forall x, y, z \in X, (x, y) \in A \wedge (y, z) \in A \Rightarrow (x, z) \in A$.
- 6) Асимметричность: $\forall x, y \in X, (x, y) \in A \Rightarrow (y, x) \notin A$.
- 7) Эти свойства определяют тип отношения (например, эквивалентность, порядок, толерантность).

20) Толерантность, эквивалентность, отношения порядка.

- 1) Толерантность: отношение, которое рефлексивно и симметрично, но не обязательно транзитивно.
- 2) Эквивалентность: отношение, которое рефлексивно, симметрично и транзитивно; разбивает множество на непересекающиеся классы эквивалентности.
- 3) Отношение порядка: отношение, которое рефлексивно, антисимметрично и транзитивно (частичный порядок); если к этому добавляется свойство, что для любых $x, y \in X$ либо $(x, y) \in A$, либо $(y, x) \in A$, то это линейный порядок.

21)

22) Булевы (переключательные) функции. Способы задания булевых функций.

Булевы (переключательные) функции — логические функции, реализуемые комбинационными схемами через логические элементы или узлы (дешифраторы, мультиплексоры). Способы задания с примерами для функции $f(x_1, x_2, x_3)$, равной 1 на наборах (001, 010, 100, 110):

- 1) Числовое представление: номера наборов, где функция равна 1. Пример: $f = \vee (1, 2, 4, 6)$.
- 2) Совершенная дизъюнктивная нормальная форма (СДНФ): дизъюнкция конъюнкций для наборов, где функция равна 1. Пример: $f = \bar{x}_1 \bar{x}_2 x_3 \vee \bar{x}_1 x_2 \bar{x}_3 \vee x_1 \bar{x}_2 \bar{x}_3 \vee x_1 x_2 \bar{x}_3$.

- 3) Карта Карно: таблица, где клетки показывают значения функции. Пример: для x_1, x_2, x_3 , клетки $(0,0,1)$, $(0,1,0)$, $(1,0,0)$, $(1,1,0)$ содержат 1, остальные – 0.
- 4) Комплекс 0-кубов: список наборов, где функция равна 1. Пример: $K^0(f) = \{001, 010, 100, 110\}$.
- 23) Элементарные булевы функции двух переменных.**
 Элементарные булевы функции двух переменных включают операции: отрицание ($\sim$), дизъюнкция ($\vee$), конъюнкция ($\wedge$), импликация ($\Rightarrow$), эквиваленция ($\Leftrightarrow$), сложение по модулю 2 ($\oplus$). Примеры:
- 1) Конъюнкция: $f(x, y) = x \wedge y$, 1 только при $x = 1, y = 1$.
 - 2) Дизъюнкция: $f(x, y) = x \vee y$, 1 при $x = 1$ или $y = 1$.
 - 3) Сложение по модулю 2: $f(x, y) = x \oplus y$, 1 при $x \neq y$.
- 24) Тожества булевой алгебры. Элементарные преобразования.**
 Тожества булевой алгебры:
- 1) Импликация: $x \Rightarrow y = \bar{x} \vee y$.
 - 2) Эквиваленция: $x \Leftrightarrow y = \bar{x}\bar{y} \vee xy$.
 - 3) Сложение по модулю 2: $x \oplus y = \bar{x}y \vee x\bar{y}$. Элементарные преобразования: исключение операций $\Rightarrow, \Leftrightarrow, \oplus$, упрощение формул до дизъюнктивной нормальной формы с использованием законов распределения и Де Моргана. Пример: упрощение $(x \Rightarrow y) \wedge z$ до $(\bar{x} \vee y) \wedge z$.
- 25) Специальные классы булевых функций. Линейные; самодвойственные; функции, сохраняющие 0; сохраняющие 1, монотонные. Понятие о базисе булевых функций.**
 Специальные классы булевых функций:
- 1) Линейные: представимы в виде полинома Жегалкина без членов с конъюнкциями (только $\oplus$ и переменные).
 - 2) Самодвойственные: $f(\bar{x}_1, \bar{x}_2, \dots, \bar{x}_n) = \bar{f}(x_1, x_2, \dots, x_n)$.
 - 3) Сохраняющие 0: $f(0, 0, \dots, 0) = 0$.
 - 4) Сохраняющие 1: $f(1, 1, \dots, 1) = 1$.
 - 5) Монотонные: если $x_i \leq y_i$ для всех i , то $f(x_1, \dots, x_n) \leq f(y_1, \dots, y_n)$.
 - 6) Базис булевых функций: минимальный набор функций, из которых можно построить любую булеву функцию через композицию. Примеры базисов: {И, ИЛИ, НЕ}, {И-НЕ}, {ИЛИ-НЕ}, {И, ИСКЛ. ИЛИ, 1}

(Жегалкина). Штрих Шеффера и стрелка Пирса функционально полны сами по себе.

26) **Дизъюнктивная нормальная форма.**

Дизъюнктивная нормальная форма (ДНФ) — представление булевой функции как дизъюнкции (логическое ИЛИ) элементарных конъюнкций, где каждая конъюнкция включает переменные или их отрицания, соответствующие наборам, где функция равна 1. Пример: для функции $f(x_1, x_2)$, равной 1 на наборах (01, 10), ДНФ будет $f = \bar{x}_1 x_2 \vee x_1 \bar{x}_2$. Минимальная ДНФ получается после минимизации, например, с помощью карт Карно или метода Квайна-Мак-Класки.

27) **Конъюнктивная нормальная форма.**

Конъюнктивная нормальная форма (КНФ) — представление булевой функции как конъюнкции (логическое И) элементарных дизъюнкций, где каждая дизъюнкция включает переменные или их отрицания, соответствующие наборам, где функция равна 0. Пример: для функции $f(x_1, x_2)$, равной 0 на наборах (00, 11), КНФ будет $f = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2)$.

28) **Минимизация булевых функций методом Квайна-Мак-Класки.**

Метод Квайна-Мак-Класки минимизирует булевы функции через:

- 1) Построение ДНФ и комплекса 0-кубов (наборов, где функция равна 1);
- 2) Поиск кубов наибольшей размерности путем объединения наборов, отличающихся одной координатой;
- 3) Выделение простых импликант для минимального покрытия, формирующего минимальную ДНФ.
- 4) Пример: для функции $f(x_1, x_2, x_3) = \vee (0, 2, 4, 6)$, 0-кубы (000, 010, 100, 110) объединяются в 1-кубы $\bar{x}_3(00-, 10-)$, $x_1(1-0, 1-1)$, минимальная ДНФ: $f = \bar{x}_3 \vee x_1$.

29)

30) **Не полностью определенные функции, минимизация не полностью определенных функций на картах Карно и методом Квайна-Мак-Класки.**

Не полностью определенные функции имеют не заданные значения (прочерки) на некоторых наборах аргументов. Минимизация:

- 1) На картах Карно: прочерки заполняются 0 или 1 для получения минимального покрытия единиц наименьшим числом кубов наибольшей

размерности. Пример: для $f(x_1, x_2)$ с 1 в (01), 0 в (00), прочерками в (10, 11), карта с прочерками в (10, 11) заполняется 1, давая $f = x_2$.

- 2) Методом Квайна-Мак-Класки: прочерки используются для расширения кубов, минимизируя покрытие. Пример: для $f(x_1, x_2, x_3) = \vee (001)$, 0 в (000), прочерки в остальных, объединение дает куб $\bar{x}_1 \bar{x}_2$, минимальная ДНФ: $f = \bar{x}_1 \bar{x}_2$.

31) Минимизация систем переключательных функций.

Минимизация систем переключательных функций — процесс упрощения набора булевых функций, описывающих комбинационную схему, для минимизации числа используемых элементов. Выполняется методами, такими как карты Карно или Квайна-Мак-Класки, с учетом общих импликант для всех функций системы. Пример: для системы $f_1(x_1, x_2) = \vee (01, 11)$, $f_2(x_1, x_2) = \vee (10, 11)$, карты Карно показывают, что $f_1 = x_2$, $f_2 = x_1$, общий член $x_1 x_2$ может быть выделен для оптимизации схемы.

32) Алгебра высказываний.

Алгебра высказываний — раздел логики, изучающий высказывания с значениями «истина» (1) или «ложь» (0), объединенные операциями: отрицание ($\sim$), конъюнкция ($\wedge$), дизъюнкция ($\vee$), импликация ($\Rightarrow$), эквиваленция ($\Leftrightarrow$), сложение по модулю 2 ($\oplus$). Высказывания преобразуются с использованием тождеств булевой алгебры. Пример: высказывание $(A \Rightarrow B) \wedge C$ преобразуется в $(\bar{A} \vee B) \wedge C$.

33) Реализация комбинационных схем в заданном базисе. Реализация комбинационных схем в классическом базисе («НЕ», «И», «ИЛИ»). Принципы реализации «по единицам» и «по нулям». Оценка сложности комбинационных схем.

Реализация комбинационных схем в базисе «НЕ», «И», «ИЛИ»:

- 1) По единицам: схема строится на основе СДНФ, где конъюнкции для наборов с 1 реализуются элементами «И», объединенными через «ИЛИ», с «НЕ» для отрицаний. Пример: для $f(x_1, x_2) = \vee (01, 10)$, $f = \bar{x}_1 x_2 \vee x_1 \bar{x}_2$, схема включает два «И» для $\bar{x}_1 x_2$, $x_1 \bar{x}_2$, объединенных «ИЛИ».
- 2) По нулям: схема строится на основе СКНФ, где дизъюнкции для наборов с 0 реализуются через «ИЛИ», объединенные «И», с «НЕ» для отрицаний. Пример: для той же функции $f = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2)$. Оценка

сложности: число логических элементов и уровней в схеме. Пример: для f по нулям — 5 элементов (2 «НЕ», 2 «И», 1 «ИЛИ»), 2 уровня.

34) Реализация комбинационных схем в базисе Жегалкина («И», «ИСКЛ. ИЛИ», «1»).

Реализация комбинационных схем в базисе Жегалкина использует полином Жегалкина, где функция представлена как сумма по модулю 2 (ИСКЛ. ИЛИ) конъюнкций переменных и константы 1. Пример: для $f(x_1, x_2) = \vee (01, 10)$, полином Жегалкина $f = x_1 \oplus x_2$, схема включает один элемент «ИСКЛ. ИЛИ» для x_1, x_2 .

35) Реализация комбинационных схем в базисах «И-НЕ», «2И-НЕ», оценка сложности.

1) Базис «И-НЕ»: функция реализуется через элементы NAND, эквивалентные $x \bar{y}$. Пример: для $f(x_1, x_2) = x_1 \wedge x_2$, один элемент NAND реализует функцию напрямую.

2) Базис «2И-НЕ»: использует двухвходовые NAND-элементы. Пример: для $f(x_1, x_2, x_3) = x_1 \wedge x_2 \wedge x_3$, два элемента NAND: первый для $x_1 \wedge x_2$, второй соединяет его выход с x_3 . Оценка сложности: число NAND-элементов и уровней. Пример: для $f = x_1 \wedge x_2$, 1 элемент NAND, 1 уровень; для $f = x_1 \oplus x_2$, требуется 4 NAND-элемента, 2 уровня.

36) Реализация комбинационных схем в базисах «ИЛИ-НЕ», «2ИЛИ-НЕ», оценка сложности.

1) Базис «ИЛИ-НЕ»: функция реализуется через элементы NOR, эквивалентные $x \bar{y}$. Пример: для $f(x_1, x_2) = x_1 \vee x_2$, один элемент NOR реализует функцию напрямую, так как $\bar{x}_1 \bar{y}_2 = x_1 \vee x_2$.

2) Базис «2ИЛИ-НЕ»: использует двухвходовые NOR-элементы. Пример: для $f(x_1, x_2, x_3) = x_1 \vee x_2 \vee x_3$, два элемента NOR: первый для $x_1 \vee x_2$, второй соединяет его выход с x_3 . Оценка сложности: число NOR-элементов и уровней. Пример: для $f = x_1 \vee x_2$, 1 элемент NOR, 1 уровень; для $f = x_1 \oplus x_2$, требуется 4 NOR-элемента, 2 уровня.

37) Реализация комбинационных схем на дешифраторах.

Комбинационные схемы на дешифраторах реализуются с использованием дешифратора, преобразующего n -входовой код в 2^n выходов, где только один выход равен 1 для каждого входного набора, и элементов «ИЛИ» для объединения нужных выходов. Пример: для $f(x_1, x_2) = \vee (01, 11)$,

дешифратор 2-в-4 формирует выходы для (00, 01, 10, 11), выходы 01 и 11 объединяются элементом «ИЛИ», давая $f = x_2$.

38) Реализация комбинационных схем на мультиплексорах.

Комбинационные схемы на мультиплексорах реализуются с использованием мультиплексора, выбирающего один из 2^n входов на основе n управляющих сигналов. Функция задается значениями на входах мультиплексора. Пример: для $f(x_1, x_2) = \vee (01, 10)$, мультиплексор 2-в-1 с x_1 как управляющим сигналом и входами $f(0, x_2) = x_2$, $f(1, x_2) = \bar{x}_2$ реализует $f = \bar{x}_1 x_2 \vee x_1 \bar{x}_2$.